MOWASLA – SQL Query Reference

Query Name	SQL Query	Purpose / Why Use
Select All Passengers	SELECT * FROM Passenger	View all registered passengers in the system
Select All Drivers	SELECT * FROM Driver	View all registered drivers with ratings
Select All Active Rides	SELECT * FROM Ride WHERE status = 'ongoing'	Monitor currently active rides in real-time
Select Completed Rides	SELECT * FROM Ride WHERE status = 'completed'	View ride history and analyze completed trips
Passenger Wallet Balance	SELECT balance FROM Wallet WHERE wallet_id = @WalletId	Check passenger's available wallet balance
Driver Rating	SELECT rating FROM Driver WHERE driver_id = @DriverId	Display driver's performance rating
Passenger Ride Count	SELECT COUNT(*) FROM Ride WHERE passenger_id = @PassengerId	Show total rides taken by passenger
Driver Ride Count	SELECT COUNT(*) FROM Ride WHERE driver_id = @DriverId	Show total rides completed by driver
Recent Rides (Passenger)	SELECT TOP 10 * FROM Ride WHERE passenger_id = @PassengerId ORDER BY date DESC	Display passenger's recent ride history ordered by date
Recent Rides (Driver)	SELECT TOP 10 * FROM Ride WHERE driver_id = @DriverId ORDER BY date DESC	Display driver's recent completed rides ordered by date
Payment History	SELECT * FROM Payment WHERE ride_id = @RideId	View payment details for specific ride

Transaction History	SELECT * FROM Transaction_Table WHERE wallet_id = @WalletId ORDER BY date DESC	Track all wallet transactions ordered by date
Ride Feedback	SELECT * FROM Feedback WHERE ride_id = @RideId	View customer feedback for ride
Driver's Vehicle	SELECT * FROM Vehicle WHERE driver_id = @DriverId	Get vehicle information for driver
Route with Locations	SELECT R.route_id, L1.street AS start_street, L2.street AS end_street FROM Route R INNER JOIN Location L1 ON R.start_location = L1.location_id INNER JOIN Location L2 ON R.end_location = L2.location_id WHERE R.route_id = @RouteId	Display route with start and end location names
Ride Full Details	SELECT R.ride_id, P.name AS passenger_name, D.name AS driver_name, R.status, R.date, R.fare FROM Ride R INNER JOIN Passenger P ON R.passenger_id = P.passenger_id INNER JOIN Driver D ON R.driver_id = D.driver_id WHERE R.ride_id = @RideId	Get complete ride information
Driver with Vehicle	SELECT D.driver_id, D.name, D.phone, D.rating, V.model, V.plate_number FROM Driver D INNER JOIN Vehicle V ON D.driver_id = V.driver_id WHERE D.driver_id = @DriverId	Driver info with vehicle details
Passenger with Wallet	SELECT P.passenger_id, P.name, P.email, P.phone,	Passenger details with

	W.balance FROM Passenger P INNER JOIN Wallet W ON P.wallet_id = W.wallet_id WHERE P.passenger_id = @PassengerId	wallet balance
Ride with Payment	SELECT R.ride_id, R.date, R.fare, R.status, PM.method, PM.amount FROM Ride R INNER JOIN Payment PM ON R.ride_id = PM.ride_id WHERE R.ride_id = @RideId	Ride payment details
Ride with Feedback	SELECT R.ride_id, R.date, R.fare, F.rating, F.comment FROM Ride R INNER JOIN Feedback F ON R.ride_id = F.ride_id WHERE R.ride_id = @RideId	Ride feedback
All Rides with Details	SELECT R.ride_id, P.name AS passenger, D.name AS driver, R.status, R.date, R.fare FROM Ride R INNER JOIN Passenger P ON R.passenger_id = P.passenger_id INNER JOIN Driver D ON R.driver_id = D.driver_id ORDER BY R.date DESC	All rides ordered by date
Top Rated Drivers	SELECT D.driver_id, D.name, D.rating, COUNT(R.ride_id) AS total_rides FROM Driver D INNER JOIN Ride R ON D.driver_id = R.driver_id GROUP BY D.driver_id, D.name, D.rating ORDER BY D.rating DESC	Highest rated drivers
Passenger Ride History	SELECT R.ride_id, D.name AS driver, V.model, R.date, R.fare, R.status FROM Ride R INNER JOIN Driver D ON	Passenger ride history

	R.driver_id = D.driver_id INNER JOIN Vehicle V ON D.driver_id = V.driver_id WHERE R.passenger_id = @PassengerId ORDER BY R.date DESC	
Driver Earnings Report	SELECT D.driver_id, D.name, COUNT(R.ride_id), SUM(R.fare) FROM Driver D INNER JOIN Ride R ON D.driver_id = R.driver_id WHERE R.status = 'completed' GROUP BY D.driver_id, D.name ORDER BY SUM(R.fare) DESC	Driver earnings
Recent Transactions	SELECT T.transaction_id, P.name, T.amount, T.type, T.date FROM Transaction_Table T INNER JOIN Wallet W ON T.wallet_id = W.wallet_id INNER JOIN Passenger P ON W.wallet_id = P.wallet_id ORDER BY T.date DESC	Recent transactions
Route Usage Statistics	SELECT R.route_id, L1.street, L2.street, COUNT(RD.ride_id) FROM Route R INNER JOIN Location L1 ON R.start_location = L1.location_id INNER JOIN Location L2 ON R.end_location = L2.location_id INNER JOIN Ride RD ON R.route_id = RD.route_id GROUP BY R.route_id, L1.street, L2.street ORDER BY COUNT(RD.ride_id) DESC	Most popular routes
Admin Login	SELECT * FROM Admin	Authenticate admin

	WHERE email = @Email AND name = @Name	
Passenger Login	SELECT * FROM Passenger WHERE email = @Email AND phone = @Phone	Authenticate passenger
Driver Login	SELECT * FROM Driver WHERE name = @Name AND phone = @Phone	Authenticate driver
Total System Rides	SELECT COUNT(*) FROM Ride	Admin dashboard total rides
Total Completed Rides	SELECT COUNT(*) FROM Ride WHERE status = 'completed'	Admin dashboard success rate
Insert New Passenger	INSERT INTO Passenger (name, phone, email, wallet_id) VALUES (@Name, @Phone, @Email, @WalletId)	Register new passenger
Insert New Driver	INSERT INTO Driver (name, phone, license_number, rating) VALUES (@Name, @Phone, @License, 5.0)	Register new driver
Create Wallet	INSERT INTO Wallet (balance, type) VALUES (0.00, @Type); SELECT SCOPE_IDENTITY();	Initialize wallet
Record Transaction	INSERT INTO Transaction_Table (wallet_id, amount, type, date) VALUES (@WalletId, @Amount, @Type, @Date)	Log wallet activity
Update Wallet Balance	UPDATE Wallet SET balance = balance + @Amount WHERE wallet_id = @WalletId	Update wallet balance
Book New Ride	INSERT INTO Ride	Create new ride

	(passenger_id, driver_id, route_id, status, date, fare) VALUES (@PassengerId, @DriverId, @RouteId, 'ongoing', @Date, @Fare)	
Update Ride Status	UPDATE Ride SET status = @Status WHERE ride_id = @RideId	Change ride status
Record Payment	INSERT INTO Payment (ride_id, amount, method, date) VALUES (@RideId, @Amount, @Method, @Date)	Log payment
Submit Feedback	INSERT INTO Feedback (ride_id, rating, comment) VALUES (@RideId, @Rating, @Comment)	Submit feedback
Update Driver Rating	UPDATE Driver SET rating = (SELECT AVG(CAST(rating AS FLOAT)) FROM Feedback F INNER JOIN Ride R ON F.ride_id = R.ride_id WHERE R.driver_id = @DriverId) WHERE driver_id = @DriverId	Recalculate driver rating